



NOMBRE: MARTINEZ HUICAB CHRISTOPHER

MATRICULA: 67374

NOMBRE DEL PROFESOR: Sergio A. Noh Puch

NOMBRE DE LA MATERIA: Concurrencia Y Paralelismo

FECHA DE ENTREGA: 28/09/2026

**NOMBRE DE LA INSTITUCIÓN: UNIVERSIDAD
AUTÓNOMA DE CAMPECHE (CAMPUS V)**

**NOMBRE DE LA CARRERA: INGENIERÍA EN
TECNOLOGÍA DE SOFTWARE**

UAC

Universidad Autónoma
de Campeche

"Del Enigma sin Albas a Triángulos de Luz"



Explicación del algoritmo CAS

Compare-and-Swap (CAS) es una instrucción atómica a nivel de hardware que realiza la siguiente operación de forma indivisible:

```
CAS(dirección, valor_esperado, valor_nuevo):
```

```
    si *dirección == valor_esperado:
```

```
        *dirección = valor_nuevo
```

```
    return true
```

```
    sino:
```

```
        return false
```

Cómo se usa para exclusión mutua (spinlock):

Una variable compartida lock actúa como candado (0 = libre, 1 = ocupado).

Para adquirir el lock, un hilo hace CAS(lock, 0, 1) en un bucle hasta que tenga éxito.

Para liberar, simplemente se asigna lock = 0.

Es un spinlock: el hilo espera activamente (busy-waiting) en lugar de dormir.

En C, usando GCC, se implementa con `__atomic_compare_exchange_n` o el built-in `__sync_bool_compare_and_swap`.

Tabla de resultados

Hilos	Puntos	Pi calculado	Tiempo (s)
2	100000000	3.14164280	0.42
4	100000000	3.14151240	0.22
8	100000000	3.14162720	0.14
16	100000000	3.14158000	0.15

Explicación del algoritmo

CAS como primitiva atómica: el hardware garantiza que la comparación e intercambio ocurre sin interrupciones, incluso con múltiples núcleos accediendo a la misma dirección de memoria.

Spinlock: la variable lock se inicializa en 0. Cuando un hilo quiere entrar a la sección crítica, ejecuta `atomic_compare_exchange_weak(&lock, &esperado=0, 1)`. Si el valor actual es 0, lo cambia a 1 y obtiene el candado. Si ya es 1 (otro hilo lo tiene), el CAS falla y el hilo reintenta en un bucle cerrado.

Sección crítica: solo un hilo a la vez puede ejecutar `puntosDentro += locales` (o `puntosDentro++`).

Liberación: `atomic_store(&lock, 0)` pone el candado en libre para que otro hilo lo adquiera.

Propiedad fundamental: CAS garantiza exclusión mutua porque la transición $0 \rightarrow 1$ es atómica: dos hilos nunca pueden ver simultáneamente 0 y ambos pasar.

Conclusión sobre el rendimiento

Escalabilidad limitada: CAS funciona bien con poca contención, pero cuando muchos hilos compiten por el mismo lock, el cache line que contiene la variable lock rebota entre núcleos (cache ping-pong), degradando el rendimiento.

Con acumulador local (una sola entrada a la sección crítica por hilo), el overhead de CAS es despreciable y el speedup es casi lineal hasta el número de núcleos.

Con `puntosDentro++` por punto, la contención es brutal: ~78.5M de adquisiciones del lock. El tiempo crece con el número de hilos en lugar de disminuir, porque el spinlock serializa todo.

Comparado con otros algoritmos:

Test-and-Set es esencialmente el mismo spinlock, con rendimiento similar.

Peterson / Dekker solo funcionan para 2 hilos y son más costosos (múltiples escrituras por intento).

Lamport (Bakery) escala mejor en teoría pero tiene más overhead por iteración.

CAS es el más simple y rápido cuando la contención es baja, pero sufre mucho bajo alta contención.

Recomendación: usar CAS con acumuladores locales (reducción) para minimizar el número de accesos a la sección crítica, o cambiar a `atomic_fetch_add` (que internamente usa CAS/lock xadd) y eliminar el lock explícito.